[ ]:

4 Cap Weighted Portfolio Plots –> Run full factor pipeline –> Two factor regressions FMB

[15]:
```
start_date = "1999-12-01"
```

4.1 Filter all returns to match start date

[16]:
```
asset_returns_trimmed = asset_returns.loc[start_date:]
caps_df_trimmed = caps_df.loc[start_date:]
factor_returns_trimmed = factor_returns.loc[start_date:]
betas_ebc_trimmed = betas_ebc.loc[start_date:]
betas_cw_ebc_trimmed = betas_cw_ebc.loc[start_date:]
```

[49]:
```
n_q1, n_q2 = 1,3
```

[50]:
```
cw_results = fmb_pipe.run_full_factor_pipeline(
    beta_EBC_df = betas_ebc_trimmed,
    beta_CW_EBC_df=betas_cw_ebc_trimmed,
    returns_df=asset_returns_trimmed,
    caps_df = caps_df_trimmed,
    EBC_returns_df=factor_returns_trimmed["EBC"],
    Cap_EBC_returns_df=factor_returns_trimmed["CW-EBC"],
    cap_weights=True,
    n_q1 = n_q1,
    n_q2 = n_q2,
    frequency = 'daily',
    formation_frequency='monthly',
    min_assets_per_cell=3,
    f1_name="EBC",
    f2_name="CW-EBC"
)

cw_results["mean_grid"]
```

```
Q1 buckets:   0%|          | 0/1 [00:00<?, ?it/s]

/opt/anaconda3/lib/python3.12/site-packages/statsmodels/tsa/tsatools.py:162:
FutureWarning: The behavior of array concatenation with empty entries is
deprecated. In a future version, this will no longer exclude empty items when
determining the result dtype. To retain the old behavior, exclude the empty
entries before the concat operation.
  x = pd.concat(x[::order], axis=1)
/opt/anaconda3/lib/python3.12/site-packages/statsmodels/tsa/tsatools.py:162:
FutureWarning: The behavior of array concatenation with empty entries is
deprecated. In a future version, this will no longer exclude empty items when
```

```
determining the result dtype. To retain the old behavior, exclude the empty
entries before the concat operation.
  x = pd.concat(x[::order], axis=1)
/opt/anaconda3/lib/python3.12/site-packages/statsmodels/tsa/tsatools.py:162:
FutureWarning: The behavior of array concatenation with empty entries is
deprecated. In a future version, this will no longer exclude empty items when
determining the result dtype. To retain the old behavior, exclude the empty
entries before the concat operation.
  x = pd.concat(x[::order], axis=1)
```

[50]:

	1	2	3
1	0.000328	0.000428	0.000557

[51]:
```
rename_map =  {
    "Q1_Q1": "High Pref",
    "Q1_Q2": "Mid Pref",
    "Q1_Q3": "Low Pref"
}
# Save pre-rename copy for true geometric mean computation (needs Q1_Q1 style␣
 ↪column names)
cw_portrets_raw = cw_results['portrets_wide'].copy()
cw_results['portrets_wide'] = cw_results['portrets_wide'].
 ↪rename(columns=rename_map)
```

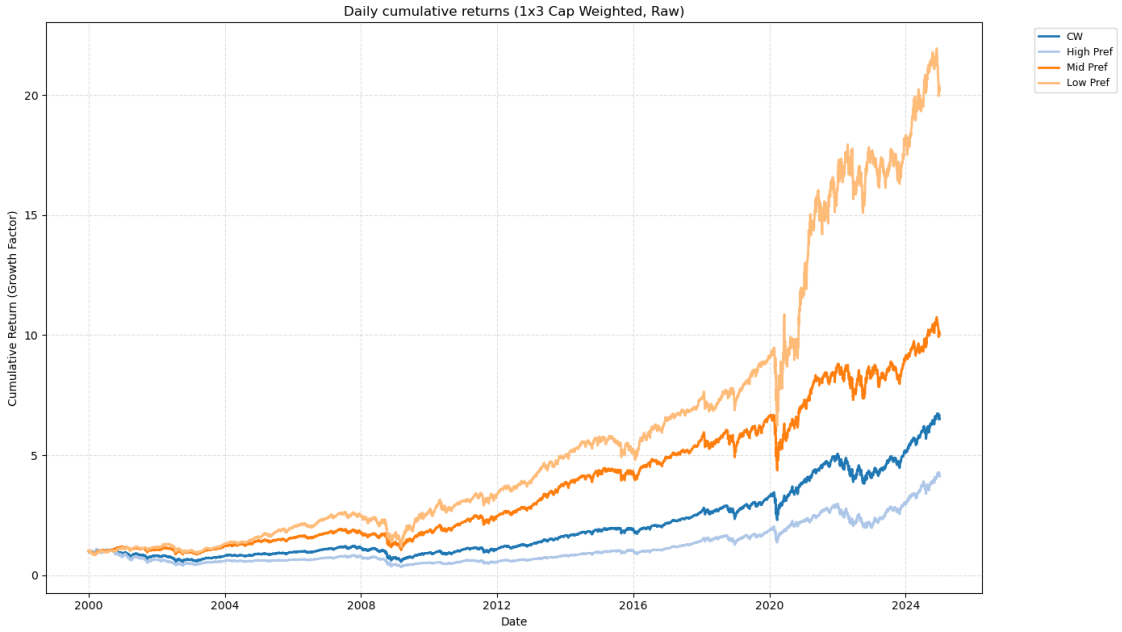
[19]:
```
rename_map = {
    "Q1_Q1": "EBC_High_Pref_High",
    "Q1_Q2": "EBC_High_Pref_Mid",
    "Q1_Q3": "EBC_High_Pref_Low",
    "Q2_Q1": "EBC_Mid_Pref_High",
    "Q2_Q2": "EBC_Mid_Pref_Mid",
    "Q2_Q3": "EBC_Mid_Pref_Low",
    "Q3_Q1": "EBC_Low_Pref_High",
    "Q3_Q2": "EBC_Low_Pref_Mid",
    "Q3_Q3": "EBC_Low_Pref_Low",
}

cw_portrets_raw = cw_results['portrets_wide'].copy()
cw_results['portrets_wide'] = cw_results['portrets_wide'].
 ↪rename(columns=rename_map)
```

[ ]:
```
#full_merged_df.index = full_merged_df.index.to_timestamp(how="start")

cw_complete_df = full_merged_df.join(
    cw_results['portrets_wide'],
    how="inner"
)
selected_columns = ['CW',
```

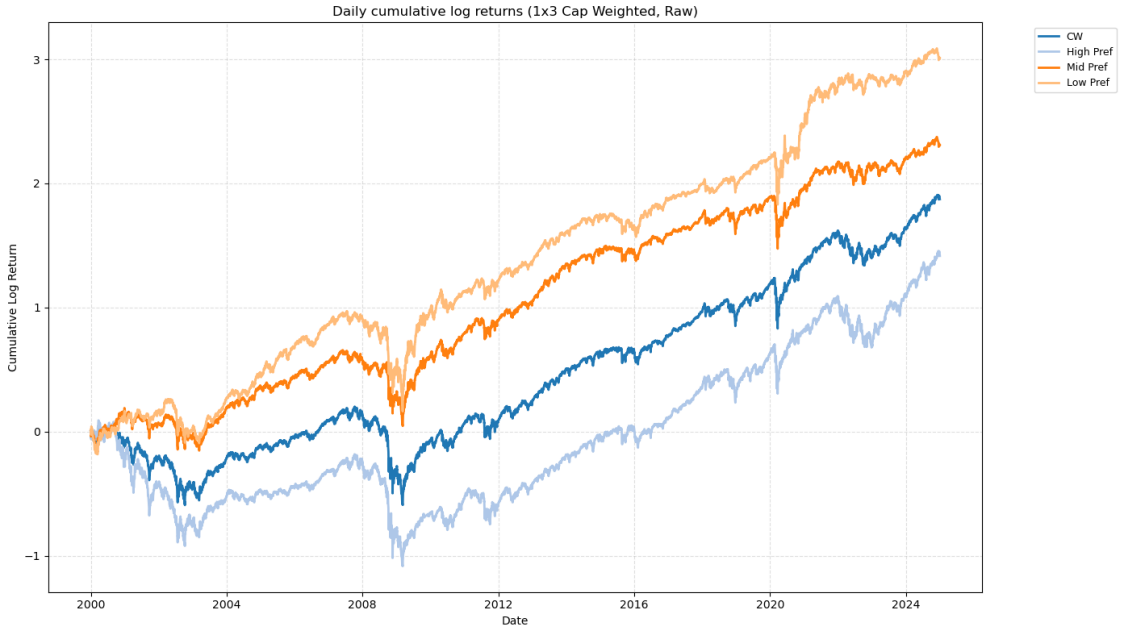
```
                    'EBC_High_Pref_High', 'EBC_High_Pref_Mid',␣
 ↪'EBC_High_Pref_Low',
                    'EBC_Mid_Pref_High', 'EBC_Mid_Pref_Mid', 'EBC_Mid_Pref_Low',
                    'EBC_Low_Pref_High', 'EBC_Low_Pref_Mid', 'EBC_Low_Pref_Low']
```

[52]:
```
cw_complete_df = full_merged_df.join(
    cw_results['portrets_wide'],
    how="inner"
)

selected_columns = ["CW","High Pref","Mid Pref","Low Pref"]
```

[53]:
```
fmb_plots.plot_cum_returns(cw_complete_df[selected_columns], title=f"Daily␣
 ↪cumulative returns ({n_q1}x{n_q2} Cap Weighted, Raw)", log=False)
```



[54]:
```
fmb_plots.plot_cum_returns(cw_complete_df[selected_columns], title=f"Daily␣
 ↪cumulative log returns ({n_q1}x{n_q2} Cap Weighted, Raw)", log=True)
```

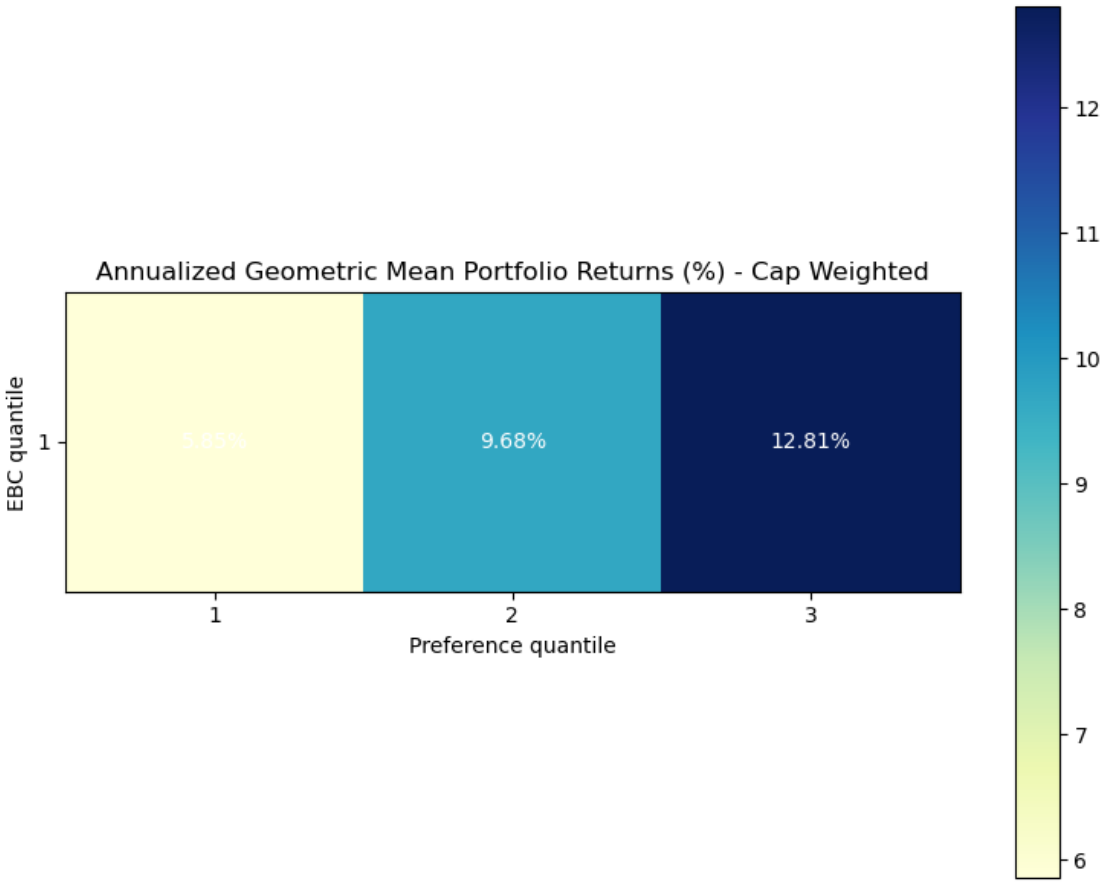


[55]:
```
period = start_date[:4] + '_2024'
save_dir = DATA_PROCESSED_DIR / 'daily'

daily_cw_quantile_returns = cw_complete_df[selected_columns]
daily_cw_quantile_returns.to_csv(save_dir /␣
 ↪f"daily_{period}_cw_quantile_returns_{n_q1}x{n_q2}.csv")
```

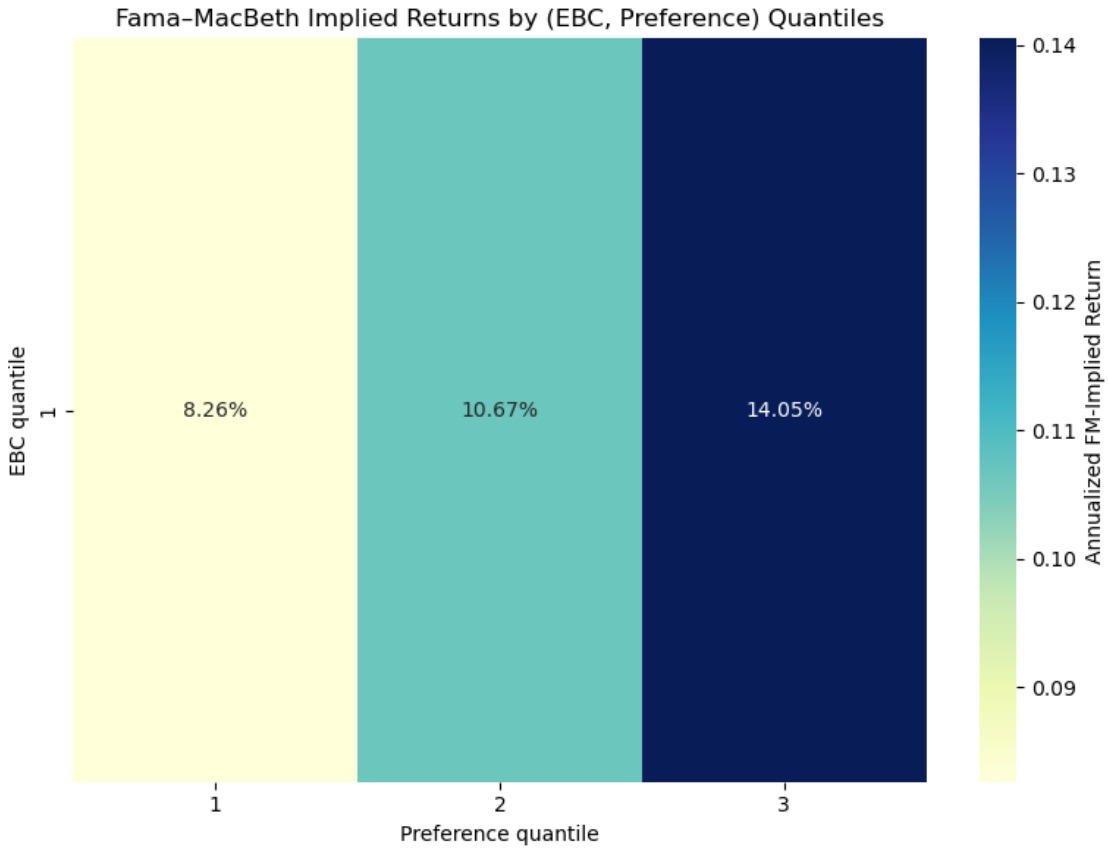
[56]:
```
mkt_geo = np.expm1(np.log1p(cw_complete_df['MKT']).mean())
mkt_ann = (1 + mkt_geo) ** 252 - 1

cw_geo = np.expm1(np.log1p(cw_complete_df['CW']).mean())
cw_ann = (1 + cw_geo) ** 252 - 1

print(f"Annualized Geometric Mean Return ({start_date[:4]}-2024)")
print(f"  CW  (ours) : {cw_ann:.2%}")
print(f"  MKT (CRSP) : {mkt_ann:.2%}")
```

```
Annualized Geometric Mean Return (1999-2024)
  CW  (ours) : 7.79%
  MKT (CRSP) : 7.94%
```

[57]:
```
# True geometric mean grid (annualized as (1+geo_daily)^252 - 1)
cw_actual_mean_grid = fmb_plots.plot_mean_grid_new(
    cw_results["mean_grid"], annualize=True, frequency='daily', cw=True,␣
 ↪geometric=True,
    portrets_wide=cw_portrets_raw
)
```



[58]:
```
cw_implied_mean_grid = fmb_plots.
↪plot_expected_return_grid(cw_results["pricing"], cw_results["fm_table"],␣
↪n_q1, n_q2,frequency = 'daily')
```



5 Equal Weighted Portfolio Plots –> Run full factor pipeline –> Two factor regressions FMB

[59]:
```
ew_results = fmb_pipe.run_full_factor_pipeline(
    beta_EBC_df = betas_ebc_trimmed,
    beta_CW_EBC_df=betas_cw_ebc_trimmed,
    returns_df=asset_returns_trimmed,
    caps_df = caps_df_trimmed,
    EBC_returns_df=factor_returns_trimmed["EBC"],
    Cap_EBC_returns_df=factor_returns_trimmed["CW-EBC"],
    cap_weights=False,
    n_q1 = n_q1,
    n_q2 = n_q2,
    frequency = 'daily',
    formation_frequency='monthly',
    min_assets_per_cell=3,
    f1_name="EBC",
    f2_name="CW-EBC"
)
```

```
ew_results["mean_grid"]*252
```

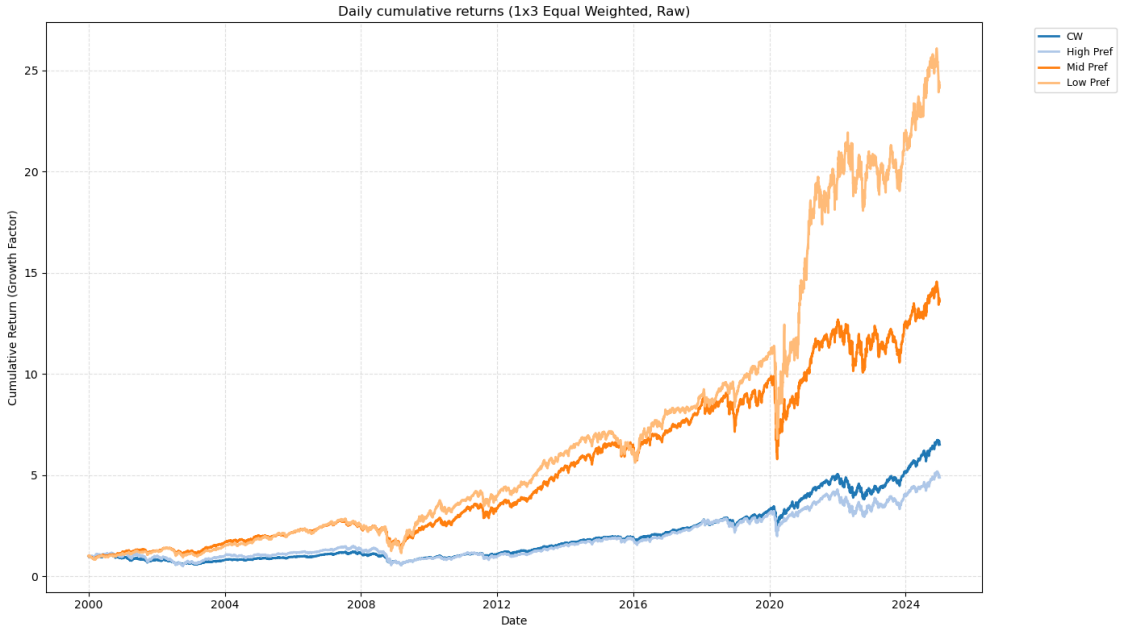
```
Q1 buckets:   0%|          | 0/1 [00:00<?, ?it/s]

/opt/anaconda3/lib/python3.12/site-packages/statsmodels/tsa/tsatools.py:162:
FutureWarning: The behavior of array concatenation with empty entries is
deprecated. In a future version, this will no longer exclude empty items when
determining the result dtype. To retain the old behavior, exclude the empty
entries before the concat operation.
  x = pd.concat(x[::order], axis=1)
/opt/anaconda3/lib/python3.12/site-packages/statsmodels/tsa/tsatools.py:162:
FutureWarning: The behavior of array concatenation with empty entries is
deprecated. In a future version, this will no longer exclude empty items when
determining the result dtype. To retain the old behavior, exclude the empty
entries before the concat operation.
  x = pd.concat(x[::order], axis=1)
/opt/anaconda3/lib/python3.12/site-packages/statsmodels/tsa/tsatools.py:162:
FutureWarning: The behavior of array concatenation with empty entries is
deprecated. In a future version, this will no longer exclude empty items when
determining the result dtype. To retain the old behavior, exclude the empty
entries before the concat operation.
  x = pd.concat(x[::order], axis=1)
```

[59]:

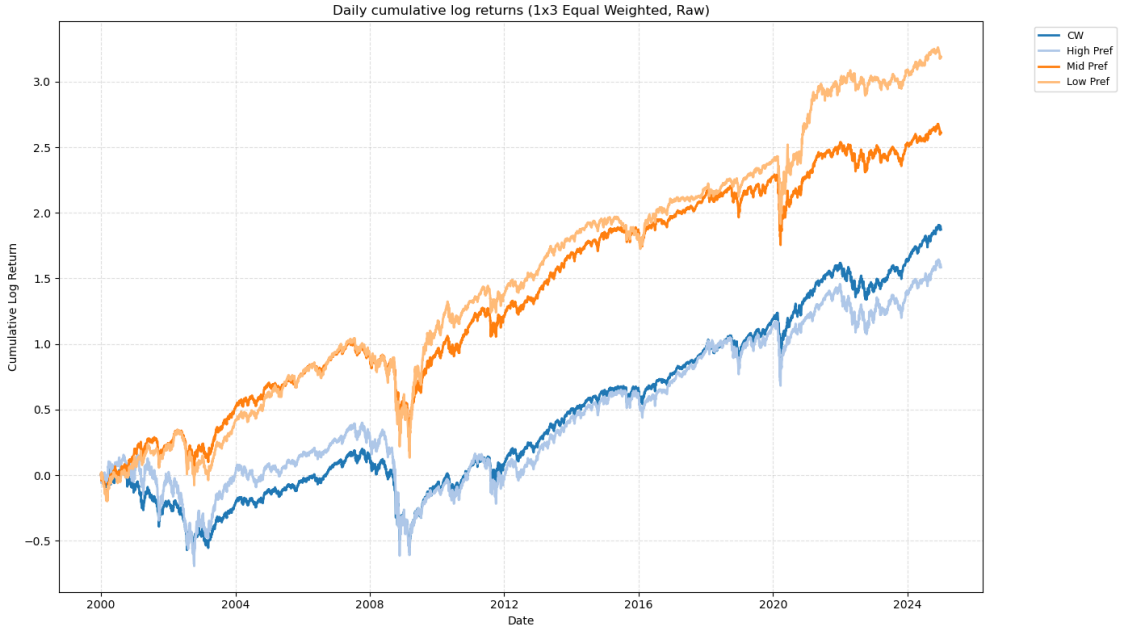
	1	2	3
1	0.093965	0.12333	0.151682

[60]:
```
# Save pre-rename copy for true geometric mean computation (needs Q1_Q1 style␣
↪column names)
ew_portrets_raw = ew_results['portrets_wide'].copy()
ew_results['portrets_wide'] = ew_results['portrets_wide'].
↪rename(columns=rename_map)
```

[61]:
```
ew_complete_df = full_merged_df.join(
    ew_results['portrets_wide'],
    how="inner"
)

fmb_plots.plot_cum_returns(ew_complete_df[selected_columns],
                                title=f"Daily cumulative returns ({n_q1}x{n_q2}␣
↪Equal Weighted, Raw)", log=False)
```



[62]:
```
fmb_plots.plot_cum_returns(ew_complete_df[selected_columns],
                           title=f"Daily cumulative log returns␣
↪({n_q1}x{n_q2} Equal Weighted, Raw)", log=True)
```

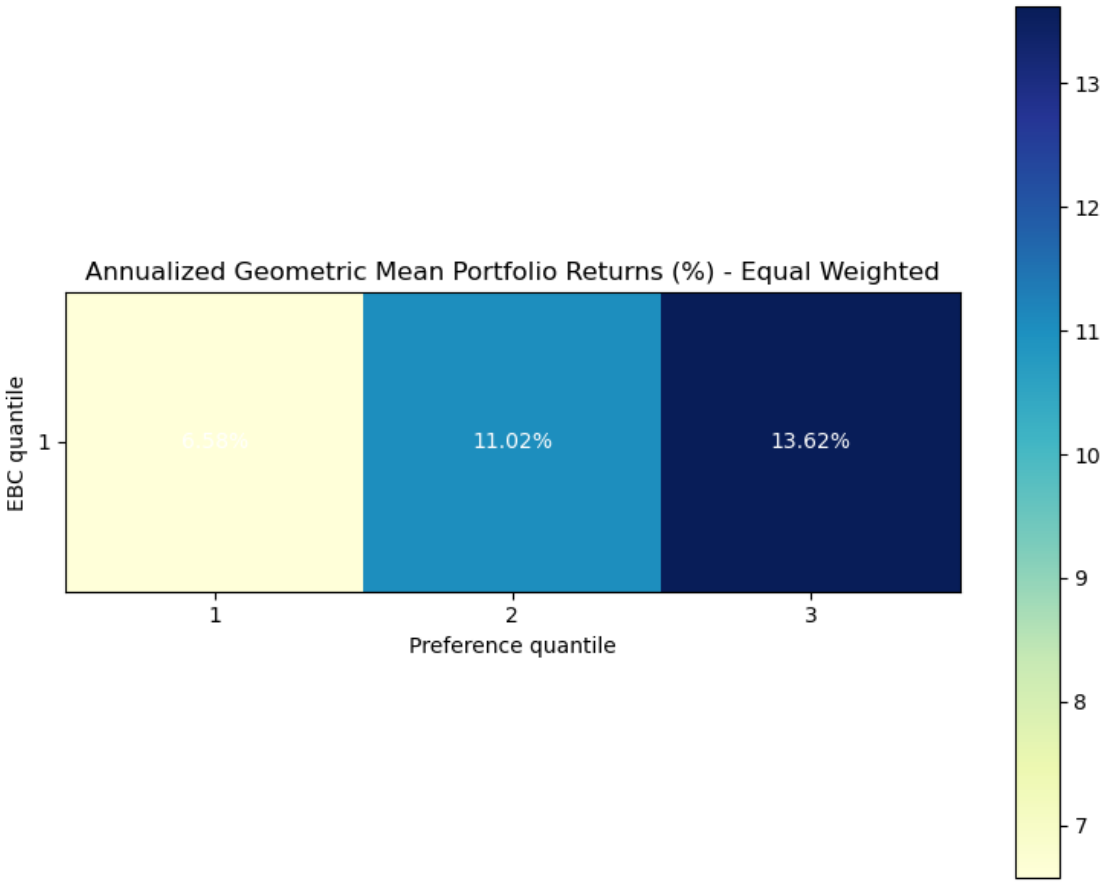


[63]:
```
daily_ew_quantile_returns = ew_complete_df[selected_columns]
period = start_date[:4] + '_2024'

save_dir = DATA_PROCESSED_DIR / 'daily'

daily_ew_quantile_returns.to_csv(save_dir /␣
 ↪f"daily_{period}_ew_quantile_returns_{n_q1}x{n_q2}.csv")
```

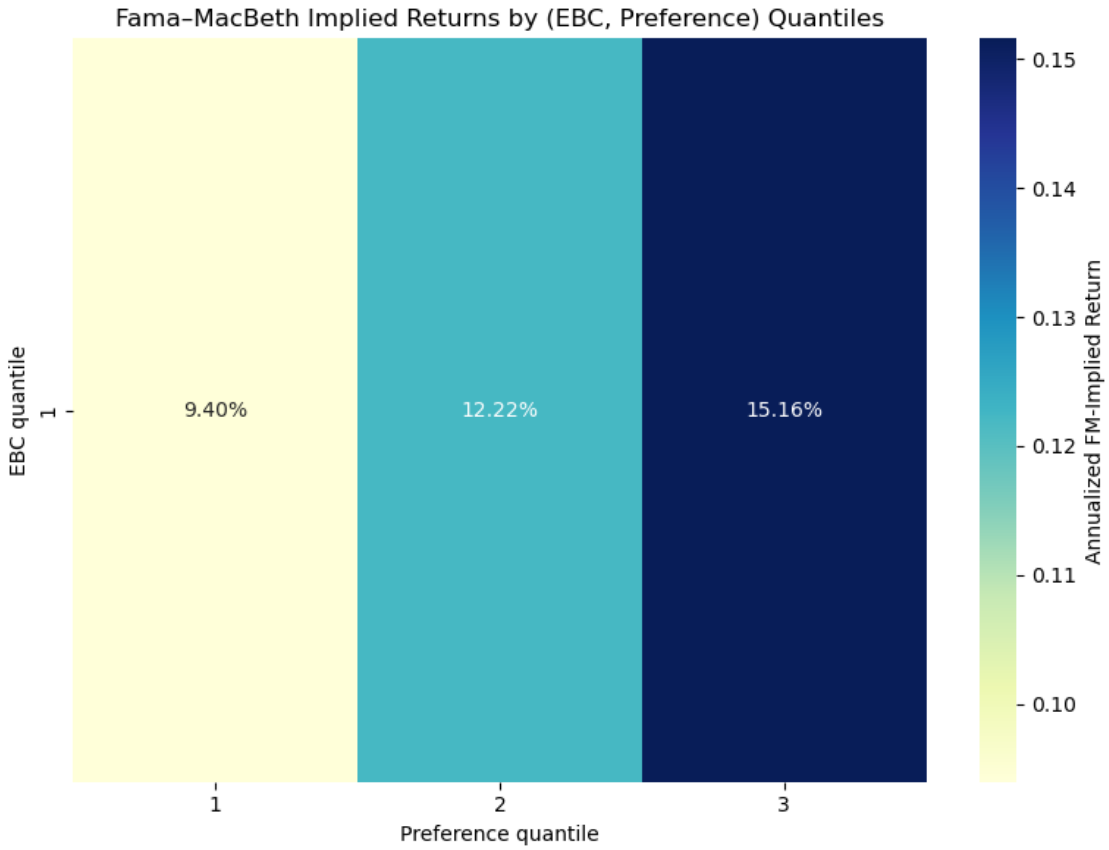
[64]:
```
#   Benchmark: annualized geometric mean returns over the analysis period
cw_geo = np.expm1(np.log1p(cw_complete_df['CW']).mean())
cw_ann = (1 + cw_geo) ** 252 - 1
print(f"Annualized Geometric Mean Return ({start_date[:4]}-2024)")
print(f"  CW (ours) : {cw_ann:.2%}")
```

```
Annualized Geometric Mean Return (1999-2024)
  CW (ours) : 7.79%
```

[65]:
```
# True geometric mean grid (annualized as (1+geo_daily)^252 - 1)
ew_actual_mean_grid = fmb_plots.plot_mean_grid_new(
    ew_results["mean_grid"], annualize=True, frequency='daily', cw=False,␣
 ↪geometric=True,
    portrets_wide=ew_portrets_raw
)
```



[66]:
```
ew_implied_mean_grid = fmb_plots.
↪plot_expected_return_grid(ew_results["pricing"], ew_results["fm_table"],␣
↪n_q1, n_q2,frequency = 'daily')
```



6 Extra Stats

[35]:
```python
# 1. Annualized Sharpe Ratio (Daily)
# We use 252 for daily and 12 for monthly
daily_rets = daily_cw_quantile_returns
sharpe = (daily_rets.mean() / daily_rets.std()) * np.sqrt(252)

# 2. Maximum Drawdown
# Calculate cumulative growth from $1
cum_rets = (1 + daily_rets).cumprod()
running_max = cum_rets.cummax()
drawdown = (cum_rets - running_max) / running_max
max_dd = drawdown.min()

performance = pd.DataFrame({
    "Ann. Return (%)": daily_rets.mean() * 252 * 100,
    "Ann. Volatility (%)": daily_rets.std() * np.sqrt(252) * 100,
    "Sharpe Ratio": sharpe,
    "Max Drawdown (%)": max_dd * 100
```